

Inspection Robot 01

From Fusion 360 CAD to a simulated ROS 2 robot:
URDF export, WSL2 + ROS 2 Jazzy setup, an indoor Gazebo world,
and CARLA dataset generation on a GPU server

Robot	Inspection Robot 01 / Horizon — 4WD skid-steer inspection platform
ROS distribution	ROS 2 Jazzy Jalisco (Ubuntu 24.04)
Simulator	Gazebo Harmonic (gz-sim 8.15)
Workspace	uni-bot → src/ubot/{ubot_description, ubot_bringup, ubot_control, ...}
Sensors	LDROBOT LD06 lidar, OAK-D Lite depth camera, RPi Camera v1, BNO055 IMU
CAD source	Inspection Robot 01.f3z (Fusion 360)
Document date	12 September 2026

What this document covers. Part 1 explains how the Fusion 360 model was restructured for URDF export and what had to be corrected afterwards. Part 2 documents the resulting ROS 2 description package. Part 3 is a from-scratch WSL2 and ROS 2 Jazzy install. Part 4 explains how to import a Gazebo world — the part that most often goes wrong on Jazzy — and documents the apartment/corridor world now wired into the simulation. Part 5 lists verification evidence and outstanding work. Part 6 documents the CARLA setup on the GPU server. Part 7 uses that simulator as an independent validation set for the occlusion score, and reports where the depth model — not the method — sets the limit.

Part 1 — Fusion 360 to URDF

1.1 Why the model had to be restructured

The Fusion URDF exporters (fusion2urdf and its forks) do not read bodies, sketches or assemblies the way a person does. They impose four hard rules, and a model that breaks any of them either fails to export or exports silently wrong:

- Every **link** must be a **top-level component** directly under the root.
- Only a component's **own bodies** are meshed. Bodies inside a sub-component are dropped from the STL but still counted in the mass — a silent error.
- The chassis component must be named exactly **base_link**.
- Every other link needs a **joint to base_link**, and the joint's **Component 2 is the parent** (so base_link goes in the second slot).

The original design was a root component holding 13 loose bodies plus linked assemblies (four wheel assemblies, an LD06 assembly, an LCD and a camera), each with the motor, holder and wheel nested *inside* the wheel component. None of that would have exported correctly.

1.2 What was changed in Fusion

Step	Action	Result
1	Created component <code>base_link</code> ; moved the 13 chassis bodies into it	Chassis becomes the root link
2	Copied each wheel out to its own top-level component; motors fixed to base, merging into base_link	Eliminates nested base_link
3	Deleted the old nested wheel assemblies	Removes duplicate geometry
4	Added 4 revolute joints on the real axle centres, <code>no rotation limits</code>	Exports as URDF <code><i>continuous</i></code> joints
5	Flattened LD06 → lidar_link, RPi cam → camera_link, LCD moved to base link	One component per link
6	Modelled the OAK-D Lite (91 × 28 × 17.5 mm, 61 g) in the front	Depth Camera represented
7	Rotated the whole robot 180° about Z	Robot faces +X, the ROS convention
8	Assigned materials: PLA 1.24 g/cm³, rubber wheels, aluminium	Realistic masses and inertias

Note. A Fusion *Revolute* joint with **no limits set** becomes a URDF **continuous** joint — which is what wheels need. If exactly one limit is set, the export aborts. Fusion has no joint type literally called “continuous”.

1.3 Two traps worth remembering

Copy/paste features stay bound to their source

Copying bodies into the new components with Fusion's copy/paste creates features that reference the original. Deleting the old wheel assemblies made three wheels jump to the origin, because the copies lost their placement transform. The fix was to rebuild each copy as an independent **base feature** (via the temporary BRep manager), which carries no dependency on the source. After that the old assemblies could be deleted safely.

The exporter shifts every joint position

The ROS 1 exporter wrote **all** joint origins and the base centre of mass shifted by a constant **(-16.4, +3.4, +19.7) mm**, while writing the meshes in the correct place. This is nearly invisible: because each mesh is drawn relative to its own (wrong) joint, the robot still *looks* right in RViz. But every wheel would spin about an axle 26 mm off-centre, and every sensor frame would be 26 mm out. The offset was recovered from the wheel mesh centres and subtracted from every joint. Link inertias were unaffected (they are stored relative to the joint, so the offset cancels).

Part 2 — The ROS 2 description package

Rather than adding a second robot package, the real CAD body replaced the box-and-cylinder placeholders inside the existing **ubot_description**. Every link and joint name was kept, so `ros2_control`, `Nav2`, the `EKF`, `SLAM Toolbox`, the `Gazebo` bridges and the `RViz` configs needed no changes at all.

2.1 Link and frame layout

Link	Position from base_footprint (m)	Notes
base_footprint	on the ground	Nav2 / EKF reference frame
base_link	0, 0, 0.066375	centred between the four wheels, chassis underside
front_left_wheel_link	0.128, 0.113, 0.033	continuous joint, axis +Y
front_right_wheel_link	0.128, -0.113, 0.033	continuous joint, axis +Y
rear_left_wheel_link	-0.128, 0.113, 0.033	continuous joint, axis +Y
rear_right_wheel_link	-0.128, -0.113, 0.033	continuous joint, axis +Y
lidar_link	0.031, 0, 0.363	LD06 centre; matches frame_id in real_robot.launch.py
camera_link	0.168, 0, 0.152	OAK-D Lite, pitched 4.09° down
rpi_camera_link	0.164, 0, 0.099	RPi cam v1, pitched 12.54° down
imu_link	-0.090, -0.046, 0.246	placeholder — needs measuring

Robot envelope: **357 × 252 × 379 mm** (length × width over wheels × height to the top of the lidar). Wheel radius 32.5 mm, wheelbase 255 mm, track 226.4 mm. Total modelled mass 2.09 kg.

2.2 Collision model

The chassis mesh has about 52,000 triangles — far too heavy to use as collision geometry. It is replaced by three stacked boxes that together enclose **100 % of the mesh vertices**:

Box	Extents in base_link (m)	Represents
Underside	x -0.168...0.168, y ±0.116, z -0.048...0	motors and holders under the chassis
Hull	x -0.178...0.179, y ±0.110, z 0...0.173	main body
Mast	x 0.010...0.112, y ±0.033, z 0.173...0.280	lidar mast

Wheels use cylinders (radius 32.5 mm, width 26.4 mm), which roll far more stably in physics than a tyre-tread mesh would. The OAK-D has **no** collision shape: it sits entirely inside the hull box and could never be touched first. Links joined by fixed joints are lumped into one rigid body, so overlapping shapes never self-collide.

2.3 Nav2 footprint correction

Both costmaps in **nav2_params.yaml** and **sim_nav2_params.yaml** used `robot_radius: 0.15`, but the robot's true half-diagonal is 0.219 m. Nav2 believed the robot was smaller than it is and would clip corners. Replaced with a rectangular footprint, which also stops it refusing gaps it genuinely fits through:

```
footprint: "[[0.185, 0.131], [0.185, -0.131], [-0.185, -0.131], [-0.185, 0.131]]"
footprint_padding: 0.015
```

2.4 A latent xacro bug, fixed in passing

ubot_base, **ubot_imu** and **ubot_ultrasonic** each defined global properties named `com_x/com_y/com_z`. Xacro properties are global and the last include wins, so `base_link` had silently been using the *IMU's* centre of mass. They are now `base_com_*` and `imu_com_*`.

2.5 Files changed

File	Change
ubot_description/meshes/inspection/*.stl	8 exported meshes (mm, scale 0.001)
urdf/body/ubot_base.urdf.xacro	real chassis mesh, base_footprint, 3 collision boxes
urdf/body/ubot_wheel.urdf.xacro	wheel meshes, joints on real axles
urdf/sensors/ubot_lidar.urdf.xacro	LD06 mesh at its true mast position
urdf/sensors/ubot_camera.urdf.xacro	camera_link is now the OAK-D Lite, 4.09° tilt
urdf/sensors/ubot_rpi_camera.urdf.xacro	new — RPi cam + optical frame, 12.54° tilt
urdf/body/ubot_robot.urdf.xacro	includes the RPi camera; ultrasonics commented out
ubot_bringup/config/sim_ubot_controllers.yaml	sim wheel_separation 0.2264, radius 0.0325
ubot_bringup/config/*nav2_params.yaml	rectangular footprint (both costmaps, both files)

Note. Real-robot wheel geometry (0.264204 / 0.033) was deliberately **not** changed. It lives in three places that must agree — ubot_controllers.yaml, ubot_ros2_control.xacro and the ESP32 firmware diff_controller.h — and on a skid-steer robot the effective wheel separation is a calibrated value, not a CAD value.

Part 3 — WSL2 + ROS 2 Jazzy on Windows

ROS 2 Jazzy targets Ubuntu 24.04. On Windows 11 the practical route is WSL2, which also runs Linux GUI applications (RViz, Gazebo) as ordinary windows through WSLg. Native Windows ROS 2 is possible but painful and is not recommended.

3.1 Install WSL2 and Ubuntu 24.04

In an **administrator** PowerShell:

```
wsl --install -d Ubuntu-24.04
```

Reboot when prompted. The first invocation usually only enables the Windows features — after the reboot, **run the same command again** to actually fetch the distribution. Verify with:

```
wsl -l -v
# 'Windows Subsystem for Linux has no installed distributions' means the distro step
# did not complete: run 'wsl --install -d Ubuntu-24.04' again (admin not required).
```

The distro install finishes at an interactive prompt asking for a UNIX username and password (the password is invisible while typing; it is the one sudo will ask for).

Starting and stopping WSL afterwards

- Start: type `wsl` in PowerShell, or open **Ubuntu** from the Start menu, or pick the Ubuntu tab in Windows Terminal.
- Leave the shell: `exit`.
- Shut the VM down completely: `wsl --shutdown` from PowerShell.

3.2 Install ROS 2 Jazzy

```
sudo apt update && sudo apt install -y software-properties-common curl
sudo add-apt-repository -y universe
```

Add the ROS 2 apt source (this is the current mechanism; the old apt-key method is gone):

```
export ROS_APT_SOURCE_VERSION=$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest \
  | grep -F "tag_name" | awk -F\" '{print $4}')
curl -L -o /tmp/ros2-apt-source.deb \
  "https://github.com/ros-infrastructure/ros-apt-source/releases/download/\
  ${ROS_APT_SOURCE_VERSION}/ros2-apt-source-${ROS_APT_SOURCE_VERSION}.${. /etc/os-release \
  && echo $VERSION_CODENAME)_all.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb
```

Then the desktop stack plus the tools this project needs:

```
sudo apt update
sudo apt install -y ros-jazzy-desktop ros-dev-tools liburdfdom-tools \
  ros-jazzy-xacro ros-jazzy-joint-state-publisher-gui \
  ros-jazzy-ros-gz ros-jazzy-gz-ros2-control ros-jazzy-ros2-controllers \
  ros-jazzy-navigation2 ros-jazzy-nav2-bringup ros-jazzy-nav2-rviz-plugins \
  ros-jazzy-slam-toolbox ros-jazzy-robot-localization ros-jazzy-twist-stamper
```

Finally make every new shell source ROS automatically:

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
echo $ROS_DISTRO      # must print: jazzy
```

Note. If `colcon build` fails with “*Could not find a package configuration file provided by `ament_cmake`*”, ROS is simply not sourced in that shell. That is what the line above fixes — it was the exact failure hit during this setup, because the `echo ... >> ~/.bashrc` step had not taken effect.

3.3 Build the workspace without copying files

WSL sees the Windows drives under `/mnt/c`, so the Windows checkout can be symlinked into a Linux-side workspace — no duplicate copies, no syncing:

```
mkdir -p ~/ubot_ws/src
ln -s "/mnt/c/Users/<you>/Documents/Chibueze/Horizon/Inspection Robot 01/uni-bot/src/ubot" \
    ~/ubot_ws/src/ubot
cd ~/ubot_ws
colcon build --packages-select ubot_description ubot_bringup
source install/setup.bash
```

3.4 Check the model

```
xacro src/ubot/ubot_description/urdf/body/ubot_robot.urdf.xacro > /tmp/ubot.urdf
check_urdf /tmp/ubot.urdf
ros2 launch ubot_description display.launch.py      # RViz opens as a Windows window
```

In RViz set **Fixed Frame** to `base_footprint` and drag the sliders in the joint-state window: each wheel must rotate on its own axle without wobbling. Enable the **TF** display to confirm the camera frames point forward and slightly down.

Note. Always rebuild after editing a URDF. `ubot_robot.urdf.xacro` pulls its parts in via `$(find ubot_description)`, which resolves to the *installed* copy under `install/share` — not your `src` tree. Without `colcon build`, RViz keeps showing the old model. Building once with `--symlink-install` makes `src` edits apply immediately.

Part 4 — Importing a Gazebo world (Jazzy / Harmonic)

This is the part that usually fails. ROS 2 Jazzy pairs with **Gazebo Harmonic** (the gz command, gz-sim 8.x) — *not* Gazebo Classic. Most tutorials and most downloadable worlds (including the popular AWS house and bookstore worlds) target Classic, use <gazebo> SDF 1.6 syntax and Classic-only plugins, and will not load. The reliable route is to take a **model** from Gazebo Fuel and write a small Harmonic world around it.

4.1 Finding a world or model

Fuel's *world* collection is thin; the good indoor environments are published as *models*. Search the REST API directly:

```
curl -s -G https://fuel.gazebosim.org/1.0/models \
  --data-urlencode 'q=house' --data 'per_page=10' | python3 -m json.tool | grep '"name'"
```

Candidates evaluated for this project:

Model (OpenRobotics)	Real size	Verdict
Apartment	20 × 22 m	chosen — two halls ~7 × 20 m joined by passages
House 1	14.6 × 12.9 m	usable, smaller rooms
Office Building	4.3 × 4.3 m	too small to drive in
Warehouse / Depot	large	aisles, but industrial rather than building interior

Note. Check the units before trusting a size. These COLLADA meshes declare <unit name="inch" meter="0.0254"/>. Read raw vertex values and the Apartment looks 797 m across; multiplied by 0.0254 it is a realistic 20 m. Some models also carry a <scale> in their model.sdf (House 1 uses 1.5).

4.2 Download and vendor the model

```
gz fuel download -u "https://fuel.gazebosim.org/1.0/OpenRobotics/models/Apartment"
# lands in ~/.gz/fuel/fuel.gazebosim.org/openrobotics/models/apartment/<version>/
```

A world can reference a Fuel model by URL and Gazebo will fetch it on first run. That was rejected here for two reasons: it needs internet on every fresh machine, and this model has a texture bug. Its COLLADA references the textures by *bare filename*:

```
[Err] [SystemPaths.cc:526] Could not resolve file [apartment_diffuse.jpg]
```

The .jpg files live in materials/textures/ while Gazebo resolves relative to the mesh directory, so the walls render untextured. The model was therefore copied into the package (2.1 MB) with a duplicate of each texture placed beside the .dae, and its internal Fuel URLs rewritten to model://apartment/...:

```
ubot_bringup/models/apartment/
  model.config
  model.sdf          # uri rewritten to model://apartment/meshes/apartment.dae
  meshes/apartment.dae
  meshes/apartment_diffuse.jpg    <- duplicated so the DAE resolves
  meshes/apartment_spec.jpg       <- duplicated so the DAE resolves
  materials/textures/*.jpg
```

4.3 The three things that break world loading

Symptom	Cause and fix
<i>Unable to find file .../meshes/x.dae</i>	The GZ_SIM_RESOURCE_PATH must contain the directory <i>containing</i> the model folder, and
World loads, robot never appears	The spawner addresses the world by name (-world). The <font face=
Everything is black	A directional "sun" does not light a building interior. Raise <scene><ambient&

4.4 The world file

ubot_bringup/worlds/apartment.sdf keeps the plugin and physics block of the original **basic.sdf**, and adds lighting, a ground plane 2 cm below the building floor (to avoid z-fighting) and the model include:

```
<sdf version='1.10'>
  <world name='apartment'>      <!-- must match the launch 'world' argument -->
    ...physics and gz-sim system plugins...
    <scene><ambient>0.7 0.7 0.7 1</ambient><shadows>true</shadows></scene>
    <light type='directional' name='sun'>...</light>
    <light type='point' name='hall_light_right'><pose>5 0 2.5 0 0 0</pose>...</light>
    <model name='ground_plane'><pose>0 0 -0.02 0 0 0</pose>...</model>
    <include>
      <uri>model://apartment</uri>
      <name>apartment</name>
      <pose>0 0 0 0 0 0</pose>
    </include>
  </world>
</sdf>
```

Remember to install the new directory, or the world will work from source and fail after install — in **ubot_bringup/CMakeLists.txt**:

```
install(
  DIRECTORY config launch worlds models
  DESTINATION share/${PROJECT_NAME}
)
```

4.5 Launch integration

`sim.launch.py` previously hard-coded both the world file and `-world` sensors. It now takes arguments, and appends the models directory to Gazebo's search path:

```
world      := apartment      # file in worlds/ AND the <world name> inside it
spawn_x    := 5.0
spawn_y    := 0.0
spawn_z    := 0.1
spawn_yaw  := 1.5708         # face down the corridor (+Y)

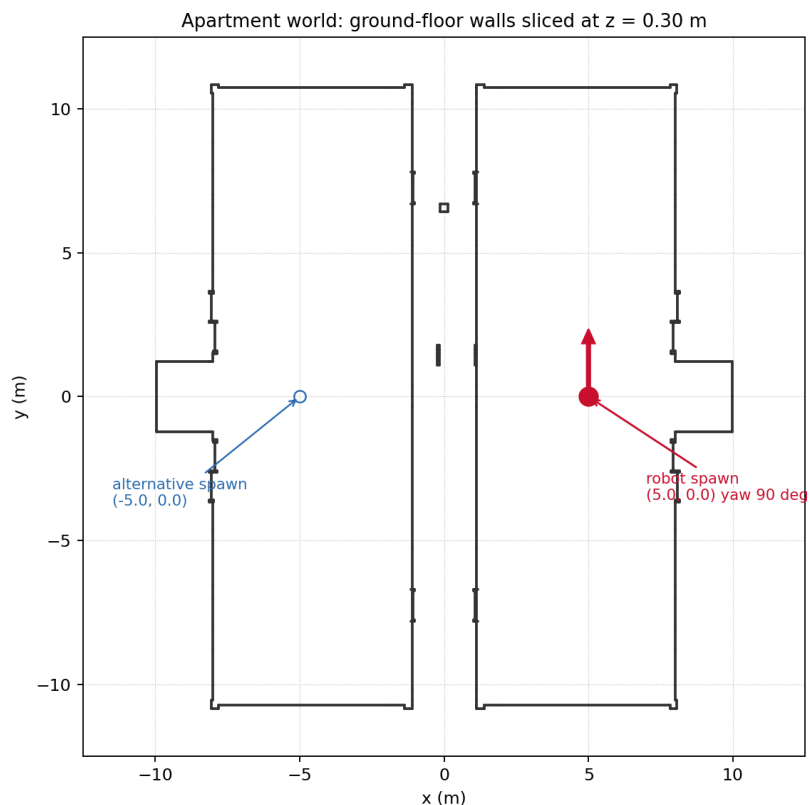
gz_resource_path = AppendEnvironmentVariable(
    'GZ_SIM_RESOURCE_PATH',
    os.path.join(get_package_share_directory('ubot_bringup'), 'models'))
```

Run the indoor world, or fall back to the old empty one:

```
ros2 launch ubot_bringup sim.launch.py
ros2 launch ubot_bringup sim.launch.py world:=basic spawn_x:=0.0 spawn_yaw:=0.0
```

4.6 Choosing a spawn point

Spawning inside a wall is the other classic failure, and in a 20 m building guessing is unreliable. The spawn point here was derived from the geometry: the collision mesh was sliced at $z = 0.30$ m (robot height) to obtain the wall segments, rasterised into a 10 cm occupancy grid, and the interior cell furthest from any wall was selected. That gives **(5.0, 0.0)** with **4.0 m** of clearance, mirrored by **(-5.0, 0.0)** in the left-hand hall. The halls run along Y, so the robot spawns with yaw = 90° facing down the corridor.



Ground-floor wall layout of the Apartment world, recovered by slicing the collision mesh at $z = 0.30$ m. Red marks the default spawn pose; blue the alternative.

4.7 Skid-steer in simulation: friction and odometry

The first full run looked fine in Gazebo but wrong in RViz: the robot appeared to rotate and drift sideways while standing still in the world. Measured against Gazebo ground truth, a commanded rotation of 0.6 rad/s produced:

Quantity	Before fix	After fix
Gazebo yaw change	0.00° (did not rotate)	103.9° in 3.0 s
Wheel velocities mid-spin	≈ 0 rad/s (stalled)	±2.09 rad/s
Wheel odometry yaw	~174° (phantom)	matches truth
actual / commanded yaw rate	0.00	1.007

Cause 1 — lateral friction stalls the wheels

A 4-wheel skid-steer can only turn by scrubbing its tyres sideways. The wheels carried $\mu_1 = \mu_2 = 1.1$, i.e. as much grip sideways as forward, so the robot was effectively bolted to the floor: the wheels stalled while `diff_drive_controller` kept integrating the commanded rotation. The odometry therefore reported a rotation that never happened — visible only in RViz, because Gazebo shows the truth.

```
<!-- ubot_gazebo.urdf.xacro, wheel_gazebo macro -->
<mu1>1.0</mu1>      <!-- along fdir1 = rolling direction: keep traction -->
<mu2>0.05</mu2>    <!-- sideways: MUST be low so the tyres can scrub -->
<fdir1>1 0 0</fdir1>
```

Cause 2 — odometry must use the *EFFECTIVE* wheel separation

Once it could turn, the robot still rotated at only **0.871** of the commanded rate, because scrubbing tyres lose rotation. Feeding `diff_drive_controller` the geometric track (0.2264 m) therefore over-reported rotation by about 15 %. The effective value is the geometric one divided by that ratio:

```
effective_separation = 0.2264 / 0.871 = 0.260 m
```

Set as `wheel_separation: 0.2600` in `sim_ubot_controllers.yaml`, the measured ratio becomes 1.007. Notably this lands within 2 % of the **0.264204 m** the real robot was independently calibrated to by driving and spinning it — mutual confirmation that the number is real physics, not a fudge.

Note. This is why the real robot's `wheel_separation` must stay a **measured** value and not be “corrected” to the CAD track. On a skid-steer the two are legitimately different.

How to measure it yourself

Publish to `/diff_drive_controller/cmd_vel` with **sim-time** stamps (wall-clock stamps make every command look stale, producing an endless “*Velocity command timed out. Braking.*”), then compare ground truth against odometry:

```
gz model -m ubot -p                                # ground-truth pose
ros2 topic echo /diff_drive_controller/odom
# drive a known distance / rotation and compare the two
```

4.8 The duplicate Sensors system crash

Symptom, seen on a second project (Occlusion) that reuses this robot in a Sonoma Raceway world: the world opened fine on its own, the robot spawned, and a few seconds later the Gazebo window *closed by itself*. Everything downstream then failed — controller_manager disappeared, and the controller spawners died with lock and switch_controller timeouts, which made it look like a controller problem.

The log told the real story. The GUI reported Received signal[2] (SIGINT) and shut down *cleanly*; it had not crashed. That is the gz wrapper shutting the GUI down because the **server** died. Running the server headless produced the backtrace:

```
Object ".../libgz-sim-sensors-system.so", at 0x..., in
Segmentation fault (Address not mapped to object [(nil)])
```

Cause

The Sensors system was loaded twice. The world declared it:

```
<plugin filename="gz-sim-sensors-system" name="gz::sim::systems::Sensors">
```

and the robot brings its own copy in `ubot_gazebo.urdf.xacro`, which loads when the robot is spawned. Two render managers in one process, and it segfaults shortly after the second one appears. The apartment world in this document never declared it, which is exactly why the same robot has always worked there — the robot's copy is the only instance.

Fix

Declare the Sensors system **exactly once**. The smaller change was to remove it from the world, leaving the robot self-contained and matching the working apartment case. Re-enable it at world level only if the world itself gains sensors *and* the robot's copy is removed.

Note. A world that opens fine alone but dies seconds after the robot appears is the signature. Check for this before suspecting the model, the spawn pose or the controllers — all three produce misleading secondary errors once the server is gone.

4.9 Simulation speed is rendering-bound, not physics-bound

With the crash fixed, the raceway world ran but /scan arrived at **2.1 Hz** instead of its configured 10 Hz — real-time factor about **0.2**, i.e. five times slow motion. The same world with no robot in it steps at RTF 1.0, so physics is not the problem: the cost is the depth camera and lidar re-rendering the scene every cycle.

Measuring the candidate worlds explains why the raceway is so expensive — its mesh covers the surrounding landscape, not just the track:

World	Extent of the mesh	Vertices
Sonoma Raceway	47.9 km × 41.9 km × 2.6 km (!)	32,559
Parking garage	60 × 42 × 46 m	17,398
Gas Station	18 × 26 × 8 m	8,467
Warehouse	244 × 100 × 4 m	1,980
Apartment (this document)	20 × 22 m	4,852

Two levers

1. Cut sensor cost. A sensor_profile argument now switches between nominal sensors and a cheap profile — the camera alone drops from 307k pixels at 30 Hz to 77k at 10 Hz, roughly twelve times less pixel work:

Profile	Lidar	Camera	IMU
full (default)	720 @ 10 Hz	640 × 480 @ 30 Hz	100 Hz
fast	360 @ 5 Hz	320 × 240 @ 10 Hz	50 Hz

2. Use a smaller world. For a robot that drives at walking pace among a handful of vehicles, a bounded scene is both cheaper and more useful than an open racetrack.

Is it the hardware?

Largely yes. The test machine renders through WSL on an **Intel UHD Graphics 620** integrated GPU with no `/dev/dri` passthrough, so Gazebo's OpenGL work goes through a translation layer. Physics is fine — 8 CPU cores hold RTF 1.0 unaided. A discrete GPU, and ideally native Linux rather than WSL, changes the rendering picture entirely; the same world can be expected to run several times faster. Note that low RTF is a speed problem, not a correctness one: physics and sensors stay consistent in *simulation* time. It only hurts interactive driving and anything tied to wall-clock.

Part 5 — Verification and outstanding work

5.1 What was actually tested

Check	Method	Result
Package builds	colcon build (ubot_description, ubot_bringup)	pass
URDF valid	xacro + check_urdf	parses; correct tree under base_footprint
Mesh paths	\$(find ...) expansion + file existence	all 8 resolve
Geometry	meshes transformed through the TF chain	max error 0.007 mm
Wheels on ground	lowest vertex in base_footprint	0.0000 m
Collision coverage	mesh vertices tested against the 3 boxes	154,860 / 154,860 inside
Live TF	tf2_echo on the running robot_state_publisher	matches CAD; tilts 4.09° / 12.54°
World loads	gz sim headless, 300 iterations	no errors or warnings
Robot spawns	ros_gz_sim create in the apartment	entity created; models: ground_plane, apartment, ubot
Lidar sees walls	/scan via ros_gz_bridge	720 beams, 275 hits, nearest 3.22 m
Straight-line odometry	truth vs /diff_drive_controller/odom	1.620 m vs 1.620 m; 0 sideways
Rotation odometry	yaw rate ratio, 2 trials	1.007 after the skid-steer fix
Sensors crash	headless server, backtrace	duplicate Sensors system; gone after the fix
Sim speed	/scan rate vs configured rate	2.1 Hz vs 10 Hz in the raceway world (RTF ~0.2)

The nearest-wall distance of 3.22 m is the strongest single check: it is exactly half the width of a 6.5 m hall, confirming the robot is standing in open floor rather than inside geometry.

5.2 Not yet done

#	Item	Why it matters
1	Recalibrate wheel geometry on the new body: drive 1 m, spin 360° to update	robot_controllers.yaml, ubot_ros2_control.xacro <div>

5.3 Command reference

```
# build (always after editing a URDF)
cd ~/ubot_ws && colcon build --packages-select ubot_description ubot_bringup
source install/setup.bash

# model only, in RViz
ros2 launch ubot_description display.launch.py

# full simulation in the apartment world
ros2 launch ubot_bringup sim.launch.py

# drive it
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# inspect
ros2 run tf2_tools view_frames          # writes a TF tree PDF
```

```
ros2 topic hz /scan
gz model --list                                # what is in the running world
```

5.4 Provenance and licensing

- Apartment model: **OpenRobotics**, Gazebo Fuel, **CC-BY 4.0** — attribution required if this world is redistributed.
- Robot meshes: exported from *Inspection Robot 01.f3z*, original work.
- Backups of the pre-change ubot_description and bringup config: `Inspection Robot 01/_backups/` (outside the workspace so colcon does not see a duplicate package).
- Session notes for continuing this work live in `uni-bot/CLAUDE.md`.

Part 6 — CARLA on the GPU server

The Gazebo simulation above works, but it is limited by the laptop's integrated GPU and by how few pedestrian and vehicle models Fuel offers. For occlusion-robust pedestrian detection the simulator's real job is to produce **images with depth and occlusion labels**, and for that CARLA on a proper GPU is a far better fit. This part documents that setup end to end.

6.1 The machine

Host	toyibat@66.172.10.85 (riftvm), Ubuntu 24.04, headless
GPU	2 × Tesla V100-SXM3 32 GB, driver 580.159.03
CPU / RAM	10 cores, 167 GB
CARLA	0.9.16, packaged build in ~/carla/0.9.16, no root required
Client	~/carla/venv, Python 3.12, carla==0.9.16 from PyPI
Measured	10.3 ticks/s with 3 cameras at 1280×720 in synchronous mode

Note. 0.9.16, not 0.9.15. The 0.9.15 package only ships Python **2.7 and 3.7** clients, which on a Python 3.12 host would have meant installing a whole conda environment just to get an old interpreter. 0.9.16 ships a cp312 wheel and is also on PyPI, so client and server match natively. Client and server versions must always agree.

6.2 Running the server

The server renders offscreen — no display, no X server:

```
cd ~/carla/0.9.16
setsid nohup ./CarlaUE4.sh -RenderOffScreen -nosound -carla-rpc-port=2000 \
  > ~/carla/server.log 2>&1 < /dev/null &

pgrep -fc CarlaUE4          # expect 4 processes
ss -ltn | grep 200          # expect 2000 and 2001 listening
pkill -f CarlaUE4           # stop it
```

Note. Use `setsid nohup ... < /dev/null &`. Started without it, the server dies the moment the SSH session closes — which looks exactly like a crash when the next client call times out.

6.3 Generating the dataset

generate_occlusion_dataset.py builds scenes, captures synchronised RGB + depth + instance frames, and writes KITTI-format labels. The camera rig matches the robot's OAK-D Lite as measured in CAD: **0.152 m** above ground, pitched **4.09°** down, **69°** horizontal FOV — so the synthetic images share the real robot's viewpoint.

```
cd ~/carla
setsid nohup ./venv/bin/python generate_occlusion_dataset.py \
  --towns Town01,Town02,Town03,Town04,Town05 --frames 600 \
  --out ~/carla/datasets/occl_v1 > ~/carla/gen.log 2>&1 < /dev/null &

tail -f ~/carla/gen.log
```

Useful switches: `--frames-per-scene` (default 10), `--occluded-fraction` (default 0.5, the share of pedestrians placed directly behind a vehicle), `--cam-height` and `--cam-pitch` (override the robot geometry — at 0.152 m roughly half the frame is road surface, which is honest for this robot but wasteful if you only want pedestrian pixels), and `--seed` for reproducibility.

6.4 What comes out

Path	Contents
image_2/000000.png	RGB, 1280×720

Path	Contents
depth/000000.png	ground-truth depth, 24-bit CARLA encoding
instance/000000.png	instance segmentation: R = semantic tag, id = G + 256×B
label_2/000000.txt	KITTI format: type truncation occluded alpha x1 y1 x2 y2
raw/000000.json	exact visible pixels, amodal + visible boxes, distance, visibility
dataset_meta.json	frame and instance counts, fill factor, occlusion histogram

Depth is ground truth, not estimated. The pipeline currently derives depth with MiDaS; here the true depth comes free, so it can be used directly and also to measure how much MiDaS error costs the fusion model — an ablation KITTI cannot provide.

6.5 How occlusion is measured

The occlusion label is measured from the instance mask rather than estimated:

- **visible_pixels** — counted exactly: pixels whose semantic tag is 12 (pedestrian) and whose decoded instance id matches that actor.
- **amodal box** — the pedestrian's 3D bounding box projected into the image, i.e. the full extent including the hidden part.
- **fill** — a silhouette factor calibrated *from the run itself*: the median visible_pixels / box area over pedestrians that are fully in frame and unoccluded. A person does not fill their own bounding box, and this measures by how much instead of assuming.
- **visibility** = visible_pixels / (fill × box area), clipped to [0, 1]; the KITTI level follows (0 below 0.2 occluded, 1 below 0.6, 2 above).

Both the calibrated fill factor and the number of samples behind it are recorded in dataset_meta.json, so the labels can be audited or recomputed from raw/*.json without re-rendering anything.

6.6 The first dataset (occl_v1)

Frames	600 — 120 each from Town01 through Town05
Pedestrian instances	2,534 labelled
Occlusion split	700 clear (28%), 751 partly (30%), 1,083 largely occluded (43%)
Distance	1.2 m to 163 m, median 13.6 m
Fill factor	0.633, calibrated from 537 unoccluded samples in this run
Generation time	17.8 min on one V100 (~1.8 s per frame)
Size on disk	1.1 GB (RGB 814 MB, depth 244 MB, instance 24 MB)

The distribution is deliberately hard: **43% of instances are largely occluded**, where KITTI's own hard tier is comparatively thin. That is the point — the metric this research optimises is AP on the hard tier, and this dataset can be regenerated with a different --occluded-fraction or seed whenever more of a given regime is needed.

6.6 Viewing the results on your own machine

Copy a sample back

```
# a handful of frames to look at
mkdir -p ~/occl_sample && cd ~/occl_sample
scp toyibat@66.172.10.85:~/carla/datasets/occl_v1/image_2/00000*.png' .
scp toyibat@66.172.10.85:~/carla/datasets/occl_v1/label_2/00000*.txt' .

# or the whole dataset (resumable, ~1.5 GB for 600 frames)
rsync -avP toyibat@66.172.10.85:~/carla/datasets/occl_v1/ ./occl_v1/
```

Draw the labels over the images

draw_check.py renders the amodal box in green, the visible box in red, and the visibility figure per pedestrian — the quickest way to confirm labels are sane:

```
# on the server
cd ~/carla && ./venv/bin/python draw_check.py ~/carla/datasets/occl_v1
scp toyibat@66.172.10.85:~/carla/datasets/occl_v1/check/*.jpg' .
```

Drive CARLA interactively from your laptop

The client is only RPC traffic — rendering stays on the V100 — so you can forward the port and run client code locally:

```
ssh -L 2000:localhost:2000 -L 2001:localhost:2001 toyibat@66.172.10.85

# then locally, in a venv with: pip install carla==0.9.16
python -c "import carla; w=carla.Client('127.0.0.1',2000); print(w.get_server_version())"
```

Note. Keep the client version equal to the server version. A mismatched client usually connects and then fails in confusing ways.

6.7 Simulating the ubot: paired occluded / fully-visible capture

ubot_capture.py goes further than the static rig: it simulates the robot's own perception and captures each pedestrian *twice* — once as the robot actually sees them, and once with the occluders removed.

Why there is no robot body in the world

CARLA is vehicle-centric and has no small inspection robot; importing one means rebuilding CARLA from source in the Unreal Editor. That costs nothing here, because the OAK-D sits on the front face looking forward — the chassis is *behind* the lens and never appears in its own images. A camera rig at the right pose sees exactly what the robot sees. What is lost is physical presence (collisions), which dataset generation does not need.

The rig

rgb	1280×720, 69° HFOV, at 0.152 m, pitched 4.09° down
mono_left / mono_right	same optics, ±37.5 mm — the OAK-D Lite's 75 mm stereo baseline
depth	ground truth, aligned with rgb
instance	per-actor masks, the basis of the labels
ego	kinematic: the rig advances along a road route between frames

Exact visibility, not an estimate

For every pedestrian in view the scene is re-rendered with all other actors teleported away, giving that pedestrian's **unoccluded silhouette**. Visibility is then a direct ratio:

```
visibility = visible pixels (normal view) / visible pixels (isolated view)
```

No fill factor, no bounding-box approximation. 84% of instances in the run below carry an exact measurement; the remainder are pedestrians with zero visible pixels, which are dropped anyway. A separate **clean_rgb** pass stores the same instant with every occluder removed, so each frame has a fully-visible counterpart for training or evaluation.

Result: ubot_v1

Frames	590 across Town01–Town05, five image streams each
Pedestrian instances	3,798 labelled
Occlusion split	2,126 clear (56%), 464 partly (12%), 1,208 largely occluded (32%)
Visibility	3,187 exact (84%), median 93%
Distance	1.3 m to 300 m, median 16 m
Size	3.4 GB

6.7 Traps worth remembering

Symptom	Cause
Client times out after the SSH session ends	Server not detached. Use <code>setsid nohup ... && /dev/null &&</code>
Pedestrians vanish from the frame; distances read as 0	Spawning a walker teleports walkers onto the navigation mesh when started. The general rule is: spawn a walker teleports walkers onto the navigation mesh when started. The general rule is: spawn a walker teleports walkers onto the navigation mesh when started.
Vehicles hover above the road	Spawning at a guessed z with physics disabled. Snap to a road waypoint, let physics settle, then spawn.
Instance ids never match spawned actors	The id is G + 256×B, not G×256+B, and the semantic tag lives in R.
Server aborts (signal 6) partway through a map	Destroying and respawning the sensor rig every frame destabilises the server. Spawn it once per map.
Server aborts while switching maps	One client loading several towns in a row reliably crashes it. Run one town per server lifetime.
Client times out; everything downstream fails	A client that dies while the world is in synchronous mode leaves the server waiting for ticks forever.
No pedestrian pixels at all	Check the semantic tags present in R: if 12 is absent, the walkers are not in frame at all — a placeholder.

Part 7 — Validating the occlusion score on CARLA

Parts 1–6 build a robot and a place to drive it. This part uses that simulator as a **validation set for the occlusion score** — the flag that decides whether a detected person is partly hidden, using only an RGB camera. The method is not introduced here; it is the subject of the paper. What CARLA adds is a second, independent test of it, with labels that cannot be wrong.

7.1 The method being tested

Every detected person gives two independent guesses of their distance:

- **Apparent height.** $z_{app} = f_y \cdot S / h_{px}$. A person of typical stature S looks smaller the further away they are.
- **Metric depth.** z_{dep} , the nearest surface inside their box, from a monocular depth model.

If nobody is in front of them, the nearest surface *is* them and the two agree. If a car is in front, depth latches onto the car while the box still spans the whole person — detectors draw full-extent boxes even when someone is partly hidden. Depth reading much nearer than body size implies an occluder. The score is:

$$\text{score} = d_{min} \times h_{px} \quad (\text{LOWER} = \text{more occluded})$$

The ratio z_{dep}/z_{app} equals $(k / (f_y \cdot S)) \cdot d_{min} \cdot h_{px}$, so the focal length, the assumed stature, and the depth model's scale collapse into a single constant that ranking cancels. No camera calibration, no stature fitting, no training, no labels.

Note. Two details decide whether an implementation is testing this method or a different one. **h_{px} is the full-extent box**, not the visible box — the method does not rely on the box shrinking. And **d_{min} is the nearest surface inside the box**, not depth averaged over the person — the whole mechanism is depth latching onto the occluder. Getting either wrong inverts the sign and measures something else.

7.2 Why CARLA is worth testing on

The score is judged by AUC against occlusion labels: pick one occluded and one visible person, how often is the occluded one ranked as more occluded. That makes the **labels** the weak link — on the real robot clip, stereo labels failed their own sanity checks at 5–8 m and produced no verdict either way.

CARLA removes that failure mode. Every frame is rendered twice, once as composed and once with the occluders deleted, so a pedestrian's visible fraction is the ratio of their pixel counts between the two renders. It is **measured, not estimated**. The set also runs the camera at 0.152 m — lower than the robot's 0.19 m — so it bounds the mounting-height problem from the pessimistic side.

Property	Value
Detections	3,293 (1,493 occluded, 1,800 visible)
Frames	573, median 6 detections per frame
Camera	1280×720, 69° FOV, 0.152 m high, 4.09° down
Occlusion label	visibility < 0.8, from clean-render pixel ratio
Depth under test	Depth Anything V2 Metric Indoor Large, RGB only

7.3 Result with ground-truth depth

This is the ceiling: what the score achieves when depth is perfect. The control is distance alone, ranked the same way — far people are occluded more often, so a score can look good by quietly preferring small boxes.

	AUC
score = $d_{min} \times h_{px}$	0.863
CONTROL: distance alone	0.644

	AUC
CONTROL: box height alone	0.638

Repeated within distance bands, where distance cannot help:

Band	n	score	distance control
0–5 m	154	0.979	0.506
5–10 m	668	0.902	0.585
10–15 m	687	0.869	0.562
15–25 m	1277	0.829	0.509
25–60 m	507	0.764	0.481

The control sits at chance inside every band while the score stays far above it. Same structure as the KITTI result, on different scenes, a different camera and exact labels. The bands run lower at range than KITTI's because this set extends to 60 m.

7.4 Result with monocular depth (RGB only)

Replacing ground truth with Depth Anything V2 costs surprisingly little overall:

Depth source	AUC (all)	0–5 m	5–10 m	10–15 m	15–25 m	25–60 m
ground truth	0.863	0.979	0.902	0.869	0.829	0.764
monocular, raw	0.819	0.969	0.895	0.839	0.797	0.739

The method survives the condition it actually has to work in. What follows is why the obvious correction to the depth model makes this worse rather than better.

7.5 The depth model is not affine-wrong, it saturates

Fitting the model against truth on each person's own surface gives:

$$z_{\text{dep}} = 0.297 \cdot z + 5.541$$

A 5.5 m offset, and $\text{correlation}(\text{score}, \text{distance}) = -0.714$ — within noise of the -0.63 measured on the robot. Different camera, different checkpoint, same pathology, so the offset finding replicates independently. But a single fit hides what is really happening:

True range	median prediction	ratio	local slope
3–5 m	4.61	1.09	0.74
5–8 m	6.74	1.01	0.67
8–12 m	8.94	0.89	0.54
12–18 m	10.60	0.70	0.25
18–25 m	11.90	0.55	0.15
25–40 m	13.16	0.48	0.00

local fit 1–6 m : $z_{\text{dep}} = 0.960 \cdot z + 0.795$
 local fit 12+ m : $z_{\text{dep}} = 0.199 \cdot z + 7.636$

True 1–60 m is compressed into **4.2–16.6 m**. The model is near-metric to about 8 m and then flat. So $z_{\text{dep}} = a \cdot z + b$ is affine over any local window, but the coefficients move with range and the gain goes to zero. It is a saturating response, not an offset.

Note. Note the ceiling. The outdoor checkpoint reads roughly 16 m for everything; the indoor one saturates at 16.6 m, just further out. Choosing the indoor model buys usable range, not an escape from saturation.

7.6 Which calibration actually helps

All three calibrations below were fitted on 287 frames and scored on the other 286, so none is fitted to its own test set.

Calibration	all	0–5 m	5–10 m	10–15 m	15–25 m	25–60 m
raw (none)	0.813	0.980	0.894	0.837	0.788	0.734
global affine	0.747	0.833	0.854	0.893	0.807	0.752
near-only affine (<8 m)	0.831	1.000	0.905	0.851	0.793	0.740
monotone inverse	0.831	0.526	0.889	0.871	0.800	0.739

The global affine makes things worse — 0.813 to 0.747. Fitted across the whole range it is dominated by the saturated far instances, so it divides near-range depth by 0.297 and amplifies its noise, destroying the band the score was strongest in. Fitting on **near instances only** gives $z_{\text{dep}} = 0.897 \cdot z + 1.009$, close to unit gain, and beats raw everywhere.

The monotone inverse ties on aggregate but collapses at 0–5 m. The cause is structural: the response curve's domain starts at a predicted 4.7 m because the model never reports anything nearer, so every close detection clamps to one knot, the scores tie, and ranking dies. An inversion can reallocate range the model resolved; it cannot recover range the model never resolved.

Correlation with distance is the wrong success criterion

raw	-0.728	AUC 0.813	
global affine	+0.294	AUC 0.747	
near-only	-0.656	AUC 0.831	<- best detector
monotone	-0.522	AUC 0.831	

The best detector is still strongly range-correlated, and the variant nearest to decorrelated is the worst. Driving correlation(score, distance) toward zero therefore is not the objective — on the robot clip it tracked the fix only because that clip's range spread sits inside the model's near-linear region. **The within-band AUC table is the defensible evidence; the correlation figure is not.** The practical statement is that offset removal is valid inside the depth model's linear range, which for this hardware is roughly 1–10 m — likely all a robot avoiding people needs.

7.7 Does this validate the detector?

Every figure so far used **ground-truth boxes**. A deployed pipeline hands the score detector boxes, which are jittered and mis-scaled, so the numbers above are only meaningful if the score tolerates that. Boxes were perturbed by detector-grade amounts, five random trials each, on monocular depth:

Box perturbation	AUC	change
none (oracle boxes)	0.819	—
tight — 2% shift, 3% scale	0.818	–0.000
typical — 5% shift, 7% scale	0.816	–0.002
loose — 10% shift, 15% scale	0.807	–0.012
scale error only, 25%	0.788	–0.030

It barely moves. That follows from the construction: d_{min} is a percentile over the box interior, not a measurement of the box boundary, so the score does not need an accurate box — only one that contains the person and whatever is in front of them.

Nor does the result depend on where the occlusion label is drawn:

Label threshold	AUC	n occluded	n visible
visibility < 0.60	0.823	1365	1928

Label threshold	AUC	n occluded	n visible
visibility < 0.70	0.824	1429	1864
visibility < 0.80	0.819	1493	1800
visibility < 0.90	0.812	1604	1689
visibility < 0.95	0.807	1671	1622

The separation is monotone in raw numbers, without any threshold at all:

Ground-truth state	n	median score
fully visible	1622	1267
barely clipped	178	1076
partly hidden	313	765
mostly hidden	1180	660

What this does and does not establish

Validated. The mechanism is sound and robust to input noise: it separates occluded from visible well above the distance control, with estimated depth rather than perfect depth, with realistic box error, and regardless of where the label boundary sits. The labels behind those claims are measured rather than estimated, so they cannot drift.

Not established. Three gaps remain, and none of them should be papered over:

- **Appearance gap.** These are rendered images. How the depth model saturates on real photographs at this mounting height may differ. The saturation result transfers as a warning to check for, not as a measurement of the real camera.
- **Real detector failure modes.** Jitter tests box *noise*. It does not test missed detections, duplicate boxes, or one box spanning two overlapping people — that last case would corrupt `d_min` in a way jitter cannot simulate.
- **Prevalence of the degenerate case.** See 7.9: this set is crowded, so it shows the lone-person failure is real without showing how often it costs anything.

Note. The honest summary: CARLA validates that the score works and is robust. It does not replace a test on real images, and should not be presented as one.

7.8 Looking at the frames by hand

Aggregate AUC hides the individual cases. **viewer.html** at the root of the data folder opens in any browser — no server, no Python — and draws the detection boxes over each frame: red where the score flagged an occlusion, green where it did not, with a table giving each person's true visibility, true range, `d_min`, box height and score. Rows are highlighted where the score disagreed with the truth.

- **only frames with a miss** jumps straight to the failures, which is where it becomes clear whether they are real weaknesses or label edge-cases.
- **show clean render** (or the `c` key) swaps to the occluder-free version of the same frame, showing exactly what was hidden.
- Arrow keys step through all 573 frames.

The flag threshold is the best-F1 operating point on this set, score < 1061 (F1 0.741). Rebuild it after changing anything with `python3 make_viewer.py`.

7.9 The lone-person case

The deployed flag is relative: a detection is scored against the others in its frame, so one person in view is their own reference. That reproduces exactly — all **19 of 19** single-detection frames score 1.000, AUC 0.500, no

verdict. Where the flag can compare ($n = 3,274$) it reaches 0.748.

Note. This confirms the mechanism but **cannot measure how often it bites**. CARLA scenes are crowded — median 6 detections per frame — so only 3.3% of frames are degenerate, against 122 of 133 on the robot clip. The robot clip remains the evidence for prevalence; CARLA only shows the failure is real.

Judging a single person needs real metres, which is what the calibration above provides. Scoring calibrated depth against apparent-height depth directly, with a threshold taken from the calibration half:

Absolute judgement	AUC (all eval)	threshold	P	R	F1
near-only affine	0.831	0.56	0.696	0.787	0.739
monotone inverse	0.831	0.80	0.697	0.798	0.744

The result that matters: absolute judgement **matches** the ranking form at 0.831 while needing no reference detection. Nothing is given up by going absolute, and the lone-person case is recovered.

Note. The lone-frame AUC came out 1.000 for both variants. **Disregard it** — $n = 9$ with 2 occluded is far too small to mean anything.

7.10 Reproducing this

No virtualenv is needed: the ROS 2 Jazzy install already provides numpy, Pillow and matplotlib. The data exists in two places, kept in step by rsync: in WSL at `~/carla-data`, where the scripts run, and as a native Windows copy under **Inspection Robot 01**\carla-data (5.2 GB), where viewer.html and Explorer previews work.

```
cd ~/carla-data/scripts

# the score, ground-truth depth then monocular, with controls and bands
python3 occl_score_eval.py --depth-dir gt
python3 occl_score_eval.py --depth-dir ~/carla-data/datasets/ubot_v1/pseudo_depth

# saturation table
python3 sat_check.py

# calibration comparison and the lone-person case (split-half)
python3 calib_and_lone.py

# robustness: detector-grade box noise + label-threshold sweep
python3 robustness.py

# rebuild the hand-inspection viewer
python3 make_viewer.py
```

Re-sync anything from the GPU box with **bash ~/carla-data/sync_from_gpu.sh**; rsync is incremental. To test a different depth model, write per-frame .npy files in metres named to match the frames and point --depth-dir at them.

Note. judge_absolute in calib_and_lone.py is a reconstruction from the method description, not the project's own occlusion_score.py. It should be diffed against the real implementation before any number from 7.7 is quoted.

7.11 Traps worth remembering

Symptom	Cause
The score anti-correlates with occlusion	Using the visible box instead of the full-extent box, or depth averaged over the person instead of the head.
The score looks strong but so does distance	Always report the distance control, and repeat within distance bands. A score that quietly prefers distance is not a score.
Removing the depth offset makes results worse	The model saturates; a global affine is dominated by far instances and wrecks the near range. Fix the offset.
Calibration fixes the correlation but not the error	Expected. Correlation is not the objective — judge on within-band AUC.
A response-curve inversion ties all near detections	The main starts where the model's output starts. Range the model never resolved cannot be resolved.

Symptom	Cause
Metric depth is ~4× too large; $\delta 1$ near zero	Wrong checkpoint for the mounting height. Outdoor assumes a ~1.6 m car-mounted camera; this is not the case.
Every detection in a clip scores exactly 1.00	One person in view. The relative flag has no reference; use absolute judgement.
A result rests on labels that were never audited	Stereo labels failed their own sanity checks at 5–8 m on the robot clip. CARLA's are measured from the robot.